End-to-End Cloud CI/CD Pipeline

CIA 2 Technical Report - CS4003 DevOps

Student	A Sheikh Abdullah (23011102001), B.Tech CSE (IoT), 7th Semester
Date	29 October 2026
Repository	https://github.com/Sheikh162/webapp (public)
Deployment	Azure VM (k3s) - namespace sheikh-devops - NodePort 30081

Automated pipeline (GitHub Actions) for a Node.js web service: build, test, containerize, publish to GHCR, deploy to cloud Kubernetes (k3s on Azure), with Ansible configuration management and a proven commit-to-production loop.

Course: CS4003 DevOps | Assessment: CIA 2 - CI/CD Pipeline using AWS Student: A Sheikh Abdullah (23011102001), B.Tech CSE (IoT), 7th Semester Date: 29 October 2026 | Repository: https://github.com/Sheikh162/webapp (public)

1. Problem Definition

Development teams building and releasing a web application by hand repeat the same error-prone steps on every release: build the code, run tests, package a container image, push it somewhere, update the cluster, and check that the app still answers. Manual releases are slow, hard to reproduce, and a single skipped test can take down production.

This project designs and implements a complete Continuous Integration and Continuous Deployment pipeline for a web application. Every push to the main branch triggers an automated chain: source checkout, dependency install, unit tests, container image build, image publication to a container registry, deployment to a cloud-hosted Kubernetes cluster, and a live availability check. A second commit carrying a visible API change proves the loop is truly continuous: push, rebuild, redeploy, and the change appears at the application endpoint with no manual step.

As permitted by the course notes, the application is a simple full-stack Node.js

```
service (Express) rather than a Java/Maven project, so that the evaluation can focus
```

on the pipeline itself instead of application complexity.

2. Architecture

Push to GitHub (master)

```
-> GitHub Actions runner (build-test job: npm ci, npm test)
-> GitHub Actions runner (build-push job: docker build, push ghcr.io/sheikh162/webapp:<sha> + :latest)
-> GitHub Actions runner (deploy job: scp k8s/ to Azure VM, ssh kubectl apply)
-> Azure VM (Ubuntu 24.04, k3s single node)
-> namespace sheikh-devops
-> Deployment sheikh-webapp (1 replica, image pinned to <sha>)
-> Service sheikh-webapp-service (NodePort 30081 -> container 3000)
-> Verification: rollout status, pod Running, curl / and /health on :30081
```

GitHub Actions is the CI/CD platform, GHCR is the container registry, and Ansible provides configuration management for the cluster-side apply (playbook + inventory in ansible/). The application exposes GET / (JSON with status, environment, version, pipeline) and GET /health (200 OK).

3. Tool Selection and Justification

Application: Node.js 22 + Express (one dependency). Rationale: the smallest surface that still exercises every required pipeline stage. Candidates in the author's project folder were rejected for pipeline purposes: Avani needs MongoDB and Cloudinary; inboxfix, jobtwin, portfolio, metaverse-lab and network-lab are Next.js apps with databases; Trading-Platform is a multi-service monorepo; pillpal adds schedulers, IVR and billing; pillpal-prototype is a static file with nothing to build or test.

CI/CD platform: GitHub Actions over Jenkins. Jenkins needs a self-maintained controller (Java, plugins, credentials, webhooks) plus agent setup on the VM, and the project's legacy Jenkinsfile targets Windows batch syntax while the VM is Linux. Actions needs only one YAML file, runs on hosted runners, streams logs on github.com for the examiner, and reruns with one click.

Registry: GitHub Container Registry (ghcr.io). It uses the built-in GITHUB_TOKEN, so no extra credentials, and the repository is public so k3s can pull anonymously.

Orchestration: the VM already runs k3s v1.36.5 (single node, containerd). k3s is a conformant lightweight Kubernetes, ideal for a 2 vCPU / 3.8 GB VM, and kubectl applies the same manifests any cloud Kubernetes would take.

Configuration management: Ansible (ansible-core 2.19). A playbook plus inventory performs the cluster-side deployment deterministically: ensure namespace, copy manifests, apply, wait for rollout, print status.

4. Repository Layout

server.js (Express app, / and /health), tests/app.test.js (node:test, 2 tests), package.json (start + test scripts), Dockerfile (node:22-alpine, npm ci, HEALTHCHECK), .dockerignore, k8s/namespace.yaml + deployment.yaml + service.yaml,

```
ansible/inventory.ini + playbook.yml + ansible.cfg,
```

.github/workflows/ci-cd.yml, evidence/ (pipeline runs, test output, live verification).

5. Pipeline Stages and Their Purpose

The workflow (CI/CD Pipeline, on push to master/main plus manual dispatch) has three jobs. build-test checks out the code, sets up Node 22 with npm cache, runs npm ci for a reproducible install, and runs npm test (2 tests: / returns Success payload with environment, version and pipeline fields; /health returns OK). A red test stops everything before any image is built. build-push logs in to GHCR with the automatic token and builds/pushes two tags from the same layers: : for immutable deploys and :latest for convenience. deploy copies k8s/ to the VM over SCP, then over SSH ensures the sheikh-devops namespace, applies the manifests, pins the Deployment image to : and stamps APP_VERSION with the sha, waits for rollout success, prints pods and services, and curls the NodePort endpoint plus /health. Pinning the image by sha makes every deployment traceable to one commit.

6. Cloud Resources and Deployment Approach

VM: Azure, hostname DevOps-VM, Ubuntu 24.04.5 LTS, 2 vCPU (AMD EPYC 7763), 3.8 GB RAM (2.8 GB free), 29 GB disk (23 GB free), k3s v1.36.5 single node, containerd runtime. The VM had no Docker CLI, node, Java, Jenkins, Helm or Ansible; none are needed on it because builds run on GitHub runners and the VM only receives

```
kubectl apply over SSH.
```

Co-tenancy: a teammate's java-web-app runs in the default namespace on NodePort 30080. All project objects live in the dedicated sheikh-devops namespace with unique labels (app: sheikh-webapp), one small replica (128-256 MB), and NodePort 30081, so no object, port, or rollout can collide with the teammate's workload. Post-deployment checks confirmed the teammate's pod and service untouched.

GHCR package: ghcr.io/sheikh162/webapp, tags :latest and :, public.

7. Screenshots and Test Results

Unit tests (npm test, also enforced in CI): 2 tests, 2 pass, 0 fail. Pipeline runs (all green): initial full-stack commit, a Node 22 runner fix, and the redeploy demo commit; each run shows build-test, build-push and deploy jobs successful (evidence/pipeline-runs.txt, evidence/redeploy-run-jobs.json). Registry: image pushed on every green build (tags :latest + :sha). Live cluster state: deployment sheikh-webapp 1/1 available with image ghcr.io/sheikh162/webapp:62a281b703b3403dec9a8a56c1c363a187ccece2, service sheikh-webapp-service NodePort 8080:30081, pod 1/1 Running. Application response:

```
{"status":"Success","environment":"Prod","version":"62a281b...","pipeline":"github-actions"}.
Health: OK. Ansible playbook run: ok=6, failed=0.
```

8. Change and Redeploy Demonstration

Commit 62a281b added a visible pipeline: 'github-actions' field to GET / plus a matching test assertion. Pushing it triggered the pipeline automatically: tests passed, a new image (:62a281b...) was built and published, k3s performed a rolling update, and the endpoint now returns the new field with version 62a281b..., verified by curl on the VM. No manual deployment step was used. APP_VERSION is stamped from the commit sha on every deploy, so each release is identifiable at the endpoint itself.

9. Challenges Encountered and Solutions

Node version skew: tests passed on local Node 26 but the GitHub runner (Node 22) rejected node --test with a directory argument. Fixed by invoking the test file explicitly (node --test tests/app.test.js). Ansible module semantics:

```
ansible.builtin.command does not run a shell, so the namespace pipe failed; the
```

task was switched to ansible.builtin.shell, and a reserved variable name (namespace) was renamed to k8s_namespace. SCP target layout: copying the k8s directory created a nested path the apply step did not expect; fixed by copying files and validating with kubectl apply --dry-run=client on the VM (all three objects accepted). Ansible re-apply resetting the sha pin: the playbook applies base manifests, which reverted the image to :latest and APP_VERSION to 1.0.0; the pipeline state was restored with kubectl set image / set env followed by a rollout, and the endpoint confirms the pinned sha.

10. Security Considerations and Limitations

Secrets (AZURE_HOST, AZURE_USER, AZURE_SSH_KEY) live in GitHub Actions secrets, never in the repository; the SSH private key file is 600 locally and in ~/.ssh. GHCR authentication uses the short-lived GITHUB_TOKEN. Blast radius is bounded by the dedicated namespace, unique labels, and a distinct NodePort (30081; the teammate uses 30080). The VM accepts key-only SSH, so key custody matters. Outside access note: at submission time the Azure NSG does not admit inbound 30081, so the endpoint answers inside the VM (verified by curl over SSH) and from any laptop via an SSH tunnel (ssh -L 8081:localhost:30081 azureuser@VM, then http://localhost:8081/). Opening one NSG rule for TCP 30081 would expose it directly. The service itself is plain HTTP, acceptable for a lab endpoint but not for production. Limitations: single k3s node (no HA), rolling update of one replica (brief capacity of one), no TLS or ingress, no image signature verification, no staging environment or automated rollback beyond the previous ReplicaSet, and metrics-server absent so autoscaling is unproven. Production hardening would add TLS termination, private registry with pull secrets, resource quotas, network policies, and a staging gate.

11. Conclusion

The pipeline meets every CIA 2 deliverable: public source repository, automated build-test-push-deploy chain with configuration file, Kubernetes manifests, Ansible playbook and inventory, Dockerfile, unit tests, deployment evidence on a cloud VM, a proven commit-to-production loop, and this report. The application runs in namespace sheikh-devops as image ghcr.io/sheikh162/webapp:62a281b703b3403dec9a8a56c1c363a187ccece2, verified live inside the VM at http://localhost:30081/ with /health OK, deployed from commit 62a281b via a fully green GitHub Actions run. For classroom demo, open an SSH tunnel or add the NSG rule for TCP 30081 and browse the public IP on port 30081.

Appendix A - Deployment Evidence (transcripts)

Pipeline runs (gh run list)

```
completed■success■Redeploy demo: expose pipeline field in API response■CI/CD Pipeline■master■push■4084682259
completed■success■Fix test script for Node 22 runner■CI/CD Pipeline■master■push■40846482927■1m35s■2026-10-2
completed■failure■CIA2: Node webapp with tests, Docker, K8s manifests, Ansible, GitHub …■CI/CD Pipeline■mast
```

Unit tests (npm test)

```
  ✔ GET /health returns OK (4.138278ms)
✔ webapp (44.37487ms)
■ tests 2
■ suites 1
■ pass 2
■ fail 0
■ cancelled 0
■ skipped 0
■ todo 0
■ duration_ms 198.198523
```

Live verification (kubectl + curl on VM)

```
=== PODS/SVC/DEPLOY ===
NAME                                  READY   STATUS        RESTARTS   AGE   IP           NODE        NOMINATE
pod/sheikh-webapp-5c6c984d6b-25rpd    1/1     Terminating   0          67s   10.42.0.23   devops-vm   <none>
pod/sheikh-webapp-75c5cbb754-685pg    1/1     Running       0          32s   10.42.0.24   devops-vm   <none>

NAME                              TYPE       CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE     SELECTOR
service/sheikh-webapp-service     NodePort   10.43.186.196   <none>        8080:30081/TCP   6m56s   app=sheikh-

NAME                              READY   UP-TO-DATE   AVAILABLE   AGE     CONTAINERS   IMAGES
deployment.apps/sheikh-webapp     1/1     1            1           6m56s   webapp       ghcr.io/sheikh162/webap

=== APP (redeployed) ===
{"status":"Success","environment":"Prod","version":"62a281b703b3403dec9a8a56c1c363a187ccece2","pipeline":"git
=== HEALTH ===
OK

=== FRIEND APP UNTOUCHED ===
NAME                                  READY   STATUS    RESTARTS        AGE
pod/java-web-app-64ff796f8b-69gkn     1/1     Running   1 (4h48m ago)   2d15h

NAME                              TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)          AGE
service/java-web-app-service      NodePort    10.43.253.27   <none>        8080:30080/TCP   2d15h
service/kubernetes                ClusterIP   10.43.0.1      <none>        443/TCP          2d17h
```